
ITSD 82

Web Software Tools

Fundamentals of Laravel

LABORATORY ACTIVITY 7

Mass Data Seeding, Performance Optimization,
and Scalability Engineering

Section	BSIT 3C
Units	2.0 (Lecture) / 4.0 (Laboratory)
Schedule	Thursday 1:00 PM – 3:00 PM
Room	CISC Room 3

Project Title	PageTurner Online Bookstore Management System
Laboratory Type	Hands-on / Project-Based
Difficulty Level	Expert
Prerequisite	Completed Laboratory Activity 6

1. Laboratory Background

In Laboratory Activity 6, students implemented enterprise-grade data portability, automated backups, audit logging, and API rate limiting. The system now handles structured data operations with robust security and compliance features.

In Laboratory Activity 7, the system faces a production-scale challenge: the bookstore catalog must scale from thousands to 1 million+ book records while maintaining sub-second query response times. This laboratory simulates real-world scenarios where e-commerce platforms experience explosive inventory growth and must optimize for performance without compromising functionality.

Students will master high-volume data generation, database performance tuning, caching architectures, and horizontal scaling strategies essential for senior backend engineering roles.

2. Learning Objectives

At the end of this laboratory activity, students will be able to:

1. Implement mass data seeding using Laravel Factories to generate 1M+ realistic records with minimal memory footprint
2. Optimize database performance through strategic indexing, query refactoring, and execution plan analysis
3. Design caching architectures using Redis for query result caching, session storage, and rate limiting offloading
4. Implement database partitioning strategies for large table management and maintenance
5. Configure read replica architectures with automatic read/write splitting and connection pooling
6. Apply advanced Eloquent optimization techniques including lazy loading prevention, cursor pagination, and chunked operations
7. Conduct load testing and performance benchmarking to validate system behavior under concurrent access
8. Implement full-text search capabilities for large-scale catalog discovery

3. System Scope and Requirements

3.1 Existing System Context (from Laboratory Activity 6)

The PageTurner system currently supports:

- Import/export of 50K+ records via chunked queue processing
- Automated backup and maintenance scheduling
- Comprehensive audit logging with tamper-proof storage
- Tiered API rate limiting (public / standard / premium / admin)
- Role-based dashboards with real-time statistics

3.2 New Functional Requirements

3.2.1 The 1 Million Book Challenge

The system must seed 1,000,000 book records with the following constraints:

Constraint	Requirement
Memory Limit	Less than 512 MB RAM during the full seeding operation
Time Limit	Less than 10 minutes on standard hardware
Data Integrity	All foreign key relationships (categories, authors, publishers) must be valid
Data Realism	Varied formats, realistic pricing distributions, and proper ISBN-13 generation

3.2.2 Performance Benchmarks

After optimization, the system must achieve the following response time targets:

Query Type	Target	Method
Catalog Listing (100 records/page)	< 100 ms	Cursor pagination + covering index
Search by ISBN (exact match)	< 50 ms	Unique index + Redis cache
Category Filter (100K+ results)	< 150 ms	Composite index + query cache
Full-Text Search (1M records)	< 300 ms	Scout + MySQL FULLTEXT index
Export (10K record chunks)	< 30 s	Queue + lazy collection streaming

3.2.3 Advanced Scalability Features

- Database table partitioning by publication year (range partitioning)
- Materialized views for bestseller reports and inventory summaries
- Read replica integration for reporting and export offloading
- Redis-backed query caching with cache tagging and invalidation
- Full-text search indexing using Laravel Scout with database driver or Meilisearch

4. Step-by-Step Implementation Guide

Phase 1: Foundation and Factory Design (Steps 1 – 3)

Step 1 Database Schema Optimization

Before seeding, optimize the books table structure for high-volume operations. Create a migration for performance indexes:

BASH

```
php artisan make:migration optimize_books_table_indexes
```

Add the following index definitions inside the migration:

PHP

```
Schema::table('books', function (Blueprint $table) {  
  
    // Composite index for common filtering patterns  
    $table->index(  
        ['category_id', 'published_at', 'is_active'],  
        'idx_books_catalog_filter'  
    );  
  
    // Covering index for price range queries  
    $table->index(  
        ['price', 'stock_quantity', 'id'],  
        'idx_books_price_stock'  
    );  
  
    // Full-text index (MySQL 5.7+ / PostgreSQL)  
    $table->fullText(['title', 'description'], 'idx_books_fulltext');  
  
    // Index for active-book filtering  
    $table->index('is_active', 'idx_books_active');  
  
    // ISBN lookup index  
    $table->index('isbn', 'idx_books_isbn_lookup');  
});
```

Key Concept: Covering indexes include all columns needed by a query, eliminating the need to look up actual table rows (index-only scans). This can reduce catalog query time by 10x or more on large datasets.

Step 2 Design the High-Performance Book Factory

BASH

```
php artisan make:factory BookFactory --model=Book
```

Key design decisions in BookFactory.php:

- **Category ID caching** — IDs are loaded once into a static property, avoiding a database query per record.
- **Publisher pool** — 15 real-world publishers are pre-defined in a static array.
- **Format-based pricing** — A match expression assigns realistic price ranges per format.
- **Valid ISBN-13 generation** — Proper 978 prefix with modulo-10 checksum calculation.
- **Bestseller state** — A factory state for high-stock, always-active book generation.

Core definition method return array:

PHP

```
return [
    'isbn'          => $this->generateValidIsbn13(),
    'title'         => $this->faker->unique()->sentence(rand(2, 6)),
    'author'        => $this->faker->name(),
    'publisher'     => $this->faker->randomElement(self::$publishers),
    'price'         => $basePrice, // format-aware pricing
    'stock_quantity' => $this->faker->numberBetween(0, 1000),
    'category_id'   => $this->faker->randomElement(self::$categoryIds),
    'format'        => $format,
    'is_active'     => $this->faker->boolean(85), // 85% active
    'created_at'    => now(),
    'updated_at'    => now(),
];
```

Step 3 Create the 1M Seeder with Chunked Batch Insert

Critical Warning: Never use `Book::factory()->count(1000000)->create()` — this instantiates 1 million Eloquent models in memory simultaneously and will exhaust available RAM.

The MassBookSeeder uses chunked batch inserts. Key constants:

PHP

```
private const CHUNK_SIZE    = 5000; // Optimal for MySQL / PostgreSQL
private const TOTAL_RECORDS = 1000000;
```

Core seeding loop with garbage collection:

PHP

```
while ($inserted < self::TOTAL_RECORDS) {
    $batchSize = min(self::CHUNK_SIZE, self::TOTAL_RECORDS - $inserted);

    // Generate batch using make() — does NOT persist models
    $books = Book::factory()->count($batchSize)->make()->toArray();

    // Raw batch insert for maximum throughput
    DB::table('books')->insert($books);

    $inserted += $batchSize;
```

```
// Force garbage collection every 10 chunks
if ($inserted % (self::CHUNK_SIZE * 10) === 0) {
    unset($books);
    gc_collect_cycles();
}
```

Register in DatabaseSeeder.php:

```
PHP

public function run(): void
{
    $this->call([
        CategorySeeder::class,    // Must run before books
        MassBookSeeder::class,
    ]);
}
```

Execute with monitoring:

```
BASH

php artisan db:seed --class=MassBookSeeder --no-interaction

# Verify count in Tinker
php artisan tinker
>>> \Illuminate\Support\Facades\DB::table('books')->count();
```

Phase 2: Query Performance Optimization (Steps 4 – 6)

Step 4

Analyze and Optimize Critical Queries

Install debugging tools:

```
BASH

composer require --dev barryvdh/laravel-debugbar
composer require --dev beyondcode/laravel-query-detector
```

The BookRepository implements several optimization strategies:

Strategy	Explanation
Cursor Pagination	O(1) performance vs. OFFSET pagination which degrades to O(n) at 1M records
Column Selection	Use select([...]) to fetch only required columns, reducing I/O bandwidth
Relation Limiting	Use with(['category:id,name,slug']) — never eager-load full models
Cache Tagging	Redis tag-based caching enables targeted invalidation without full cache flush
Covering Index Hits	Match query column selection to index columns for index-only scans

Optimized active catalog query example:

PHP

```
return Book::select([
    'books.id', 'books.isbn', 'books.title', 'books.author',
    'books.price', 'books.stock_quantity', 'books.published_at',
    'books.category_id'
])
->with(['category:id,name,slug'])
->where('is_active', true)
->orderBy('published_at', 'desc')
->orderBy('id', 'desc') // Secondary sort for stable pagination
->cursorPaginate($perPage);
```

Step 5 Configure Redis for Multi-Purpose Caching

Configure separate Redis databases for different concerns:

BASH

```
# .env configuration
CACHE_DRIVER=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis
REDIS_CLIENT=phpredis
```

Redis database allocation in config/database.php:

PHP

```
'default' => [ ... 'database' => 0 ], // General purpose
'cache'    => [ ... 'database' => 1 ], // Query result caching
'session'  => [ ... 'database' => 2 ], // Session storage
```

Note: Cache tagging, required for category-specific invalidation, is only supported with Redis or Memcached drivers. It is not available with the file or database cache drivers.

Step 6 Implement Eager Loading and N+1 Prevention

The BookResource uses whenLoaded() to prevent accidental N+1 queries and conditionally loads the description field only on the show route:

PHP

```
// Only load description on the detail route, not in list views
'description' => $this->when(
    $request->routeIs('books.show'),
    $this->description
),

// Category loaded safely - no query if not eager-loaded
'category' => new CategoryResource($this->whenLoaded('category')),
```

Always pair resources with explicit with() calls:

PHP

```
$books = Book::select($allowedFields)
->with(['category:id,name'])
```

```
->where('is_active', true)  
->cursorPaginate(100);
```

Phase 3: Advanced Scalability Features (Steps 7 – 10)

Step 7 Database Table Partitioning

MySQL 8.0+ range partitioning by publication year divides the books table into sub-tables. Queries filtering by year will only scan relevant partitions (partition pruning).

BASH

```
php artisan make:migration partition_books_by_year
```

PHP

```
PARTITION BY RANGE (YEAR(published_at)) (  
    PARTITION p_old      VALUES LESS THAN (2000),  
    PARTITION p2000      VALUES LESS THAN (2005),  
    PARTITION p2005      VALUES LESS THAN (2010),  
    PARTITION p2010      VALUES LESS THAN (2015),  
    PARTITION p2015      VALUES LESS THAN (2020),  
    PARTITION p2020      VALUES LESS THAN (2025),  
    PARTITION p_future   VALUES LESS THAN MAXVALUE  
)
```

Benefits of partitioning:

- **Query Pruning** — Year-filtered queries scan only relevant partitions.
- **Maintenance** — Old partitions can be archived or dropped without row-by-row deletion.
- **Index Size** — Indexes are maintained per-partition, reducing memory footprint.

Step 8 Materialized Views for Reporting

Materialized views pre-compute expensive aggregate queries, enabling fast reporting without full-table scans on every request:

PHP

```
CREATE MATERIALIZED VIEW mv_bestseller_stats AS  
SELECT  
    category_id,  
    COUNT(*) as total_books,  
    AVG(price) as avg_price,  
    SUM(stock_quantity) as total_inventory,  
    COUNT(CASE WHEN stock_quantity > 500 THEN 1 END) as bestseller_count,  
    MAX(published_at) as latest_publication  
FROM books  
WHERE is_active = true  
GROUP BY category_id;
```

Schedule the refresh in routes/console.php:

PHP

```
Schedule::command('app:refresh-materialized-views')  
    ->hourly()  
    ->withoutOverlapping();
```

Step 9 Full-Text Search with Laravel Scout**BASH**

```
composer require laravel/scout
php artisan vendor:publish --provider="Laravel\Scout\ScoutServiceProvider"
```

Add the Searchable trait to the Book model and implement toSearchableArray() to control indexed fields. Set shouldBeSearchable() to only index active books:

PHP

```
public function toSearchableArray(): array
{
    return [
        'id'          => $this->id,
        'title'       => $this->title,
        'author'      => $this->author,
        'publisher'   => $this->publisher,
        'description' => $this->description,
        'category'    => $this->category?->name,
        'format'      => $this->format,
    ];
}

public function shouldBeSearchable(): bool
{
    return $this->is_active;
}
```

Import all records via queue for non-blocking indexing:

BASH

```
# Full import via built-in command
php artisan scout:import "App\Models\Book"

# Chunked import via custom command for better observability
php artisan make:command IndexBooksBatch
```

Step 10 Read/Write Splitting Implementation

Laravel natively supports read/write splitting via the read and write keys in config/database.php. Setting sticky to true ensures read-after-write consistency within a single request:

PHP

```
'read' => [
    'host' => [
        env('DB_READ_HOST_1', env('DB_HOST')),
        env('DB_READ_HOST_2', env('DB_HOST')),
    ],
],
'write' => [
    'host' => [ env('DB_HOST') ],
],
```

```
'sticky' => true,    // Ensures read-after-write consistency within request
```

Tip: Route heavy reporting queries to the read replica using `DB::connection('mysql::read')` or `Book::on('mysql::read')` to offload the primary write server.

Phase 4: Testing and Validation (Steps 11 – 12)

Step 11

Performance Benchmarking

Create the BenchmarkBookQueries Artisan command:

BASH

```
php artisan make:command BenchmarkBookQueries
```

The command runs each query N times (default: 100 iterations), performs a warmup pass, and reports average / min / max / total milliseconds. It automatically validates results against the performance targets defined in Section 3.2.2 and returns a non-zero exit code on failure, enabling CI/CD integration.

BASH

```
php artisan benchmark:books --iterations=100
```

Step 12

Load Testing with Concurrent Requests

PHPUnit-based load tests in `tests/Performance/BookCatalogLoadTest.php` simulate concurrent catalog and ISBN lookup requests:

BASH

```
php artisan test --filter=BookCatalogLoadTest
```

Tests validate:

- 50 concurrent catalog requests complete without error
- Average response time stays within defined thresholds
- Cache is populated correctly after initial requests
- All responses return the expected JSON structure

5. Advanced Features Beyond 1M Seeding

5.1 Horizontal Scaling: Database Sharding

Implement a Shardable trait that uses modulo-based routing ($\text{id} \% 4$) to assign models to one of four MySQL shards. While full infrastructure sharding requires operational work, designing the application layer now avoids expensive rewrites later.

PHP

```
trait Shardable
{
    public function getShardConnection(): string
    {
        $shardId = $this->id % 4;    // 4 shards
        return "mysql_shard_{$shardId}";
    }
}
```

5.2 Asynchronous Cache Warmup

The WarmCategoryCache job pre-loads the top 1,000 books per category into Redis on a background queue. Dispatch this job for popular categories after seeding, or on a schedule, to ensure cache hits from the very first user request.

PHP

```
public function handle(): void
{
    $books = Book::select(['id', 'title', 'author', 'price', 'stock_quantity'])
        ->where('category_id', $this->categoryId)
        ->where('is_active', true)
        ->orderBy('published_at', 'desc')
        ->limit(1000)
        ->get();

    Cache::tags(["category:{$this->categoryId}"])
        ->put("category:{$this->categoryId}:popular", $books, 7200);
}
```

5.3 Smart Cache Invalidation with Model Observers

BookObserver listens to the saved() and deleted() model events to flush relevant Redis cache tags. This keeps category caches and individual ISBN caches consistent without requiring a full cache flush.

PHP

```
public function saved(Book $book): void
{
    $this->cacheService->invalidateCatalog();
    Cache::forget("book:isbn:{$book->isbn}");
    Cache::tags(["category:{$book->category_id}"])->flush();
}
```

Register in AppServiceProvider:

PHP

```
public function boot(): void
{
    Book::observe(BookObserver::class);
}
```

5.4 Query Result Streaming for Exports

Update Lab 6 exports to implement FromQuery with WithChunkReading. A chunk size of 2,000 provides efficient streaming for 1M+ record exports via background queues without memory exhaustion.

PHP

```
public function query()
{
    return Book::query()
        ->select(['isbn', 'title', 'author', 'price', 'stock_quantity',
        'published_at'])
        ->where('is_active', true)
        ->orderBy('id');
}

public function chunkSize(): int
{
    return 2000;
}
```

5.5 Database Connection Pooling

For high-concurrency scenarios with Swoole or RoadRunner, enable persistent PDO connections:

PHP

```
'options' => [
    PDO::ATTR_PERSISTENT => true,
    PDO::ATTR_EMULATE_PREPARES => true,
],
```

5.6 Intelligent Rate Limiting with Redis

Enhance Lab 6 rate limiting with a tiered Redis-backed approach:

PHP

```
$limits = [
    'public' => 30,
    'standard' => 60,
    'premium' => 300,
    'admin' => 1000,
];

return Limit::perMinute($limits[$tier] ?? 30)
    ->by($request->user()->id ?: $request->ip());
```

6. Database Enhancements

6.1 New and Modified Tables

Table	Purpose
books	Enhanced with partitioning, full-text indexes, and covering indexes
mv_bestseller_stats	Materialized view for fast bestseller and inventory reporting
search_index_queue	Pending Scout indexing jobs (database queue driver)
query_performance_logs	Slow query logging for ongoing monitoring

6.2 Migration Requirements

- All index additions must be separate migrations to allow individual rollback
- Partitioning migrations must include data migration scripts for existing data
- Materialized views require a documented refresh strategy within migration comments

7. Testing and Validation

7.1 Seeding Performance Tests

☐	1M records seeded in less than 10 minutes
☐	Memory usage stays below 512 MB throughout the seeding operation
☐	All ISBNs are valid (checksum verification)
☐	Foreign keys reference valid category records
☐	Factory generates realistic data distributions (not all identical values)

7.2 Query Performance Tests

☐	ISBN lookup: less than 50 ms average (100 iterations)
☐	Catalog listing: less than 100 ms average (100 iterations)
☐	Category filter: less than 150 ms average (100 iterations)
☐	Full-text search: less than 300 ms average (50 iterations)
☐	No N+1 query problems detected (verified using Laravel Debugbar)

7.3 Cache Validation

☐	Repeated catalog requests serve from cache in less than 10 ms
☐	Cache invalidation works correctly on book update
☐	Redis memory usage is monitored and bounded
☐	Cache tags function correctly for category-specific invalidation

7.4 Load Testing

☐	System handles 50 concurrent catalog requests without error
☐	Rate limiting correctly throttles excessive requests
☐	Read replicas receive read traffic (verified via query logs)
☐	Queue workers process Scout indexing without backlog

7.5 Data Integrity

☐	1M records are queryable via Eloquent without timeout
☐	Export of 50K records completes via queue without memory exhaustion

[] Partition pruning works correctly (EXPLAIN shows only relevant partitions scanned)

8. Deliverables

8.1 Source Code Repository

File	Description
database/factories/BookFactory.php	Optimized factory with valid ISBN-13 generation
database/seeder/MassBookSeeder.php	Chunked batch insert seeder for 1M records
app/Repositories/BookRepository.php	Optimized data access layer with cursor pagination
app/Services/BookCacheService.php	Redis caching abstraction with tag support
app/Observers/BookObserver.php	Cache invalidation logic triggered by model events
app/Console/Commands/BenchmarkBookQueries.php	Automated performance benchmarking command
app/Jobs/WarmCategoryCache.php	Async background cache warming job
config/scout.php	Laravel Scout search configuration
database/migrations/*_optimize_books_*	Index, partition, and materialized view migrations

8.2 Configuration Files

File	Description
.env.example	Redis, read replica, and Scout environment variable templates
config/database.php	Read/write splitting and Redis connection configuration
config/cache.php	Redis tag-based cache store configuration

Submission Note: Ensure all benchmark results are captured as screenshots or terminal output logs and committed alongside source code. Include a README.md documenting hardware specifications and the achieved benchmark times.